MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 4

Dynamic programming: Dijkstra and Floyd-Warshall Algorithms

Elaborated by:
st. gr. FAF-241
Aurelian-Mihai Tihon

Verified by:
univ. assist.
Fiștic Cristofor

Chișinău – 2026

Contents

INTRODUCTION **3**

1 DYNAMIC PROGRAMMING AND ALGORITHMS **4**

1.1 Dynamic programming paradigm 4

1.2 Dijkstra algorithm 4

1.3 Floyd-Warshall algorithm 4

1.4 Theoretical complexity comparison 5

2 EMPIRICAL ANALYSIS **6**

2.1 Experimental setup 6

2.2 Sparse graph analysis 6

2.3 Dense graph analysis 7

CONCLUSIONS **8**

REFERENCES **9**

ANNEX A **10**

INTRODUCTION

The present laboratory work focuses on exploring dynamic programming as a core method for designing highly efficient algorithms. Dynamic programming involves breaking down a large algorithmic problem into overlapping sub-problems, storing the computed results, and optimizing them to deduce the global minimum or maximum range of a given query. The primary objective is to study the mechanism of dynamic programming by implementing and conducting an empirical analysis of two fundamental graph traversal algorithms: the Dijkstra algorithm and the Floyd-Warshall algorithm.

The core tasks addressed in this document include studying the paradigm of dynamic programming, utilizing a high-level programming language to build functional models of the algorithms, and gathering empirical performance data. The practical implementation tracks algorithm execution times across varying vertex counts and analyzes the specific impact of edge density on computational performance. By isolating variables through the generation of distinct sparse and dense topological graphs, a detailed graphical representation of the execution behavior is derived and evaluated. The complete source code for this laboratory work, including the interactive graph visualization and benchmarking scripts, can be found on GitHub - b2955c9.

1 DYNAMIC PROGRAMMING AND ALGORITHMS

1.1 Dynamic programming paradigm

Dynamic programming is an optimization approach used in computer science to solve complex problems by decomposing them into more manageable, self-contained sub-problems. It is an extension of the divide-and-conquer strategy but specifically targets scenarios where the sub-problems continuously overlap and are re-evaluated. To prevent redundant computation, dynamic programming actively stores intermediate results in data structures such as tables or matrices. The fundamental properties that indicate a problem can be solved using dynamic programming include the following:

- the problem possesses an optimal substructure, meaning the global optimum is built from local optimums;
- the algorithmic problem exhibits overlapping sub-problems that are repeatedly encountered during execution.

1.2 Dijkstra algorithm

The Dijkstra algorithm calculates the shortest path from a single source node to all other reachable nodes within a graph with non-negative edge weights. While frequently categorized as a greedy algorithm, it heavily utilizes the dynamic programming concept of state relaxation. The algorithm repeatedly extracts the closest unvisited node and updates the minimum distance to all its adjacent neighbors.

The relaxation operation applied dynamically during traversal is formally defined by the following mathematical formula:

$$d(v) = \min(d(v), d(u) + w(u,v)) \tag{1.1}$$

where $d(v)$ represents the tentative shortest distance from the source to vertex v, $d(u)$ represents the known shortest distance to vertex u, and $w(u,v)$ is the direct edge weight connecting node u to node v.

1.3 Floyd-Warshall algorithm

The Floyd-Warshall algorithm is a definitive example of dynamic programming applied to pathfinding. Unlike Dijkstra, which is executed from a single source, Floyd-Warshall computes the shortest paths between all possible pairs of vertices in a weighted graph. It accomplishes this by incrementally allowing each vertex in the graph to act as an intermediate stepping stone for paths connecting any other two vertices.

The algorithm builds a multi-dimensional matrix, dynamically updating the shortest distances through an iterative recurrence relation defined as follows:

$$d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right) \tag{1.2}$$

where $d_{ij}^{(k)}$ is the calculated shortest path from node i to node j utilizing only the intermediate vertices from the set $\{1, 2, ..., k\}$.

1.4 Theoretical complexity comparison

To properly evaluate the algorithms, their theoretical limits regarding execution time and memory consumption must be defined. Table 2.1 illustrates the asymptotic time and space complexities for both methods. While the Floyd-Warshall algorithm utilizes a dense matrix approach resulting in a high spatial and temporal footprint, it inherently solves for all vertex pairs. The Dijkstra algorithm is highly efficient for single-source queries, but must be executed V times to achieve an all-pairs result, which significantly alters its aggregate complexity.

Table 1.1 – Complexity limits of the analyzed algorithms

Algorithm	**Time Complexity**	**Space Complexity**	**Scope**
Dijkstra (Min-Heap)	$O(E + V \log V)$	$O(V)$	Single-Source
All-Pairs Dijkstra	$O(VE + V^2 \log V)$	$O(V^2)$	All-Pairs
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All-Pairs

2 EMPIRICAL ANALYSIS

2.1 Experimental setup

An empirical analysis was conducted using a custom Python application designed to generate unweighted and weighted cyclic graphs dynamically. To properly benchmark the theoretical efficiency limits, the graph density was configured to test the computational extremes. Graphs were generated with vertex counts (V) ranging from 25 to 250.

Two primary density modes were established for the tests. The first mode represents a sparse topological grid containing a 15% edge density configuration. The second mode consists of a heavily connected dense configuration containing an 85% edge density. The execution time of both the Floyd-Warshall algorithm and an All-Pairs Dijkstra implementation was measured in seconds using standard wall-clock precision.

2.2 Sparse graph analysis

In a sparse environment, graph traversals encounter far fewer branches per node. According to Figure 2.1, the scaling of the All-Pairs Dijkstra algorithm proves significantly more efficient than the Floyd-Warshall algorithm. Because Dijkstra utilizes a priority queue, its complexity scales with respect to the number of edges.

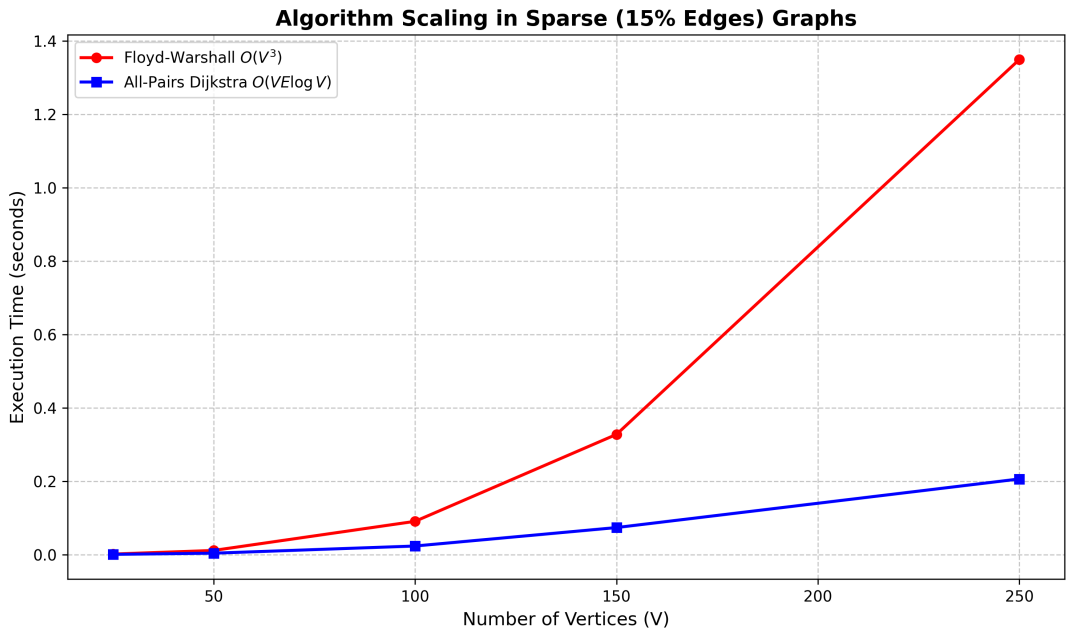

Figure 2.1 – Algorithm scaling in sparse (15% edges) graphs

The raw execution values confirming the exponential divergence between the two dynamic programming techniques in environments with limited connectivity are displayed below.

Table 2.1 – Execution times in sparse graph distributions

Nodes (V)	Floyd-Warshall	All-Pairs Dijkstra
25	0.0015	0.0007
50	0.0115	0.0039
100	0.0908	0.0235
150	0.3277	0.0738
250	1.3494	0.2062

2.3 Dense graph analysis

The dense graph variant forces the algorithms to evaluate overlapping cyclical connections continuously. According to Figure 2.2, the high edge density drastically increases the overall operational load for the priority-queue-based Dijkstra approach.

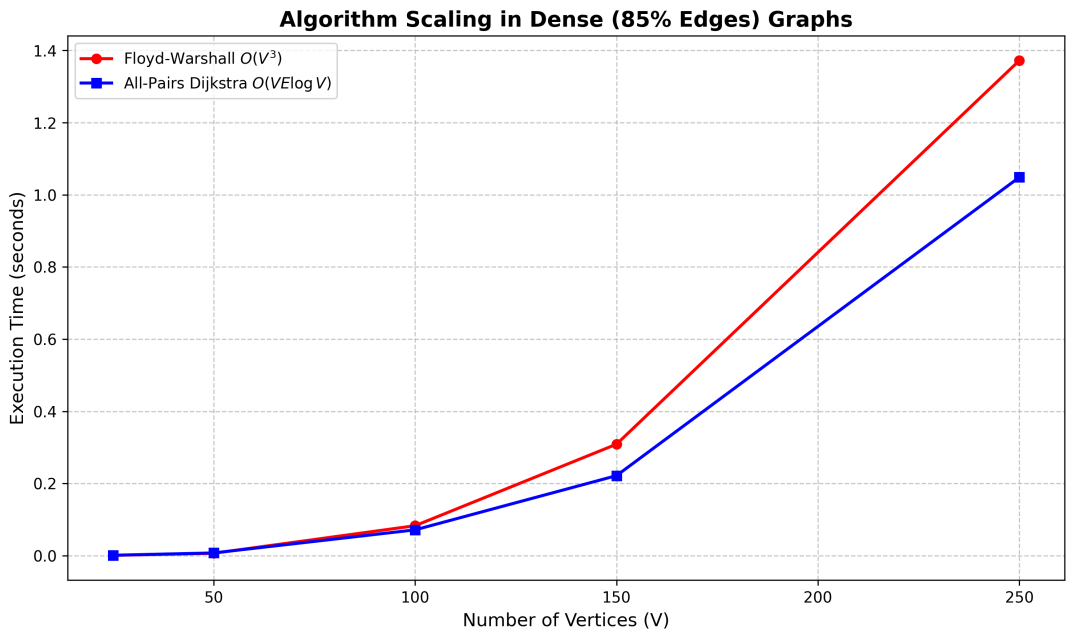

Figure 2.2 – Algorithm scaling in dense (85% edges) graphs

Despite the extreme edge density causing Dijkstra to slow down significantly compared to its sparse benchmark, the strict $O(V^3)$ loop matrix of the Floyd-Warshall algorithm remains the limiting factor. The exact comparison of empirical values is centralized below.

Table 2.2 – Execution times in dense graph distributions

Nodes (V)	Floyd-Warshall	All-Pairs Dijkstra
25	0.0011	0.0015
50	0.0070	0.0079
100	0.0833	0.0718
150	0.3091	0.2220
250	1.3720	1.0487

CONCLUSIONS

The laboratory work successfully demonstrated the practical application and empirical validation of dynamic programming principles through the implementation of the Dijkstra and Floyd-Warshall algorithms. The primary objective of bridging theoretical asymptotic bounds with real-world execution metrics was achieved. The experimental setup effectively captured the performance discrepancies between the two approaches, confirming that the structural density of the input data dictates the optimal algorithmic choice. Generating and benchmarking distinct sparse and dense topological graphs provided clear, graphical evidence of how data structures interact with execution time.

Despite the successful data collection, several technical challenges emerged during the development phase. The most difficult aspect to implement was not the core mathematical logic of the algorithms, but rather adapting them for real-time visual execution. Because the Floyd-Warshall algorithm operates strictly bounded by an $O(V^3)$ complexity format, mapping its triple-nested loops to a step-by-step generator function without freezing the main application thread proved highly complex. Maintaining the graphical state of the source, destination, and intermediate bridge nodes simultaneously required meticulous state management and careful yielding of execution frames to the rendering engine.

Furthermore, tuning the graph generation logic presented minor setbacks. Initially, dense graph generation created redundant self-loops and disconnected clusters, which heavily skewed the initial execution times for the Dijkstra algorithm. This required refactoring the generation phase to ensure a strictly connected, valid environment before the benchmarking timer was initialized. Managing the priority queue for the All-Pairs Dijkstra implementation also highlighted a critical memory management trade-off; in extreme density conditions (85% edge density), the queue operations compounded significantly, exposing the overhead of list-based heap manipulations in Python compared to the contiguous matrix array allocations used by Floyd-Warshall.

Ultimately, this empirical analysis highlights a fundamental software engineering principle - there is no universally superior algorithm. The Floyd-Warshall algorithm excels in its simplicity and predictability, solving all pairs uniformly via multi-dimensional matrices regardless of edge density, though at a steep baseline computational cost. Conversely, the Dijkstra algorithm remains the definitive choice for single-source queries and sparse networks, but it suffers severe performance penalties when forced into an all-pairs role within highly connected topologies. The laboratory work successfully solidified the understanding that an algorithm's efficiency is intrinsically tied to both its theoretical design and the specific structural nature of the data it processes.

REFERENCES

1 GeeksforGeeks, "Dynamic Programming," [Online]. Available: https://www.geeksforgeeks.org/dynamic-programming/

2 Tutorialspoint, "Dynamic Programming," [Online]. Available: https://www.tutorialspoint.com/data_structures_algorithms/dynamic_programming.htm

3 CodingNinjas, "Dijkstra's Algorithm vs Floyd-Warshall's Algorithm," [Online]. Available: https://www.codingninjas.com/codestudio/library/dijkstras-algorithm-vs-floydwarshalls-algorithm

4 T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, "Introduction to Algorithms," 3rd ed., Cambridge, MA, USA: MIT Press, 2009.

5 S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani, "Algorithms," 1st ed., New York, NY, USA: McGraw-Hill, 2006.

ANNEX A

```python
    def fw_benchmark(self, adj_matrix):
        V = len(adj_matrix)
        dist = [[adj_matrix[i][j] for j in range(V)] for i in range(V)]
        for k in range(V):
            for i in range(V):
                for j in range(V):
                    if dist[i][k] + dist[k][j] < dist[i][j]:
                        dist[i][j] = dist[i][k] + dist[k][j]
        return dist

    def all_pairs_dijkstra_benchmark(self, adj_list):
        V = len(adj_list)
        all_distances = []
        for start_node in range(V):
            distances = {node: float('inf') for node in range(V)}
            distances[start_node] = 0
            pq = [(0, start_node)]
            while pq:
                current_dist, u = heapq.heappop(pq)
                if current_dist > distances[u]: continue
                for v, weight in adj_list[u]:
                    distance = current_dist + weight
                    if distance < distances[v]:
                        distances[v] = distance
                        heapq.heappush(pq, (distance, v))
            all_distances.append(distances)
        return all_distances
```